BVS1 – Betriebssysteme

Wintersemester 2026/2027 · TH Köln · ICCT

Stavros Kousidis

Table of contents

Course Overview 4
Course Material . . . 4

1 From Executing Code to Operating Systems 5

2 From Executing Code to Operating Systems 6
2.1 1. One program can run without a guest operating system . . . 6
2.1.1 Interpreting the experiment . . . 7
2.2 2. The ISA connects software and processor . . . 8
2.2.1 Why RV64? . . . 8
2.3 3. Development host and target are different roles . . . 8
2.4 4. Virtualization and emulation are different . . . 9
2.4.1 Virtualization: the guest instructions already belong to the host ISA . . . 9
2.4.2 Emulation: the target instruction may not be a host instruction . . . 10
2.4.3 Our BVS1 machine stack . . . 10
2.5 5. The toolchain connects development host and target . . . 11
2.5.1 Object files are not yet the complete program . . . 12
2.6 6. When does an operating system become useful? . . . 12
2.7 7. Applying the working definition . . . 13
2.8 8. From Hello BVS1 to a minimal operating system . . . 13
2.9 9. Key ideas . . . 15
2.10 References . . . 15

3 From Source to Process 17
3.1 1. From source code to target-specific object code . . . 17
3.2 2. Why object files still need a linker . . . 18
3.3 3. From the stored-program idea to a concrete memory layout . . . 18
3.4 4. Linking produces the program QEMU can load . . . 19
3.5 5. Why execution starts before `main` . . . 20
3.6 6. The whole path, and what Make hides . . . 20
3.7 7. Change, rebuild, observe . . . 21
3.8 8. Why an executing program is not yet a process . . . 21
3.9 9. Yield, exit, resume, and restart . . . 22
3.10 10. The question that leads to execution context . . . 23

4 Preserving Execution 24
4.1 1. From “run” to “continue later” . . . 24
4.2 2. Return addresses make continuation visible . . . 25
4.3 3. Why a stack is needed . . . 25

4.4 4. The calling convention reduces what we need to save . . . 26
4.5 5. Represent the saved context in C . . . 26
4.6 6. The narrow assembly boundary . . . 27
4.7 7. Kernel context and process context . . . 28
4.8 8. Starting a context that has never run . . . 28
4.9 9. The bootstrap returns us to ordinary C . . . 29
4.10 10. Trace a complete cooperative switch . . . 29
4.11 11. From saved context to first process . . . 30
4.12 12. Minimal lifecycle . . . 30
4.13 13. Creation prepares the initial context . . . 31
4.14 14. Yield and resume . . . 31
4.15 15. Exit . . . 32
4.16 16. The full CP1 behaviour . . . 32
4.17 17. Key ideas . . . 33

5 I Have a Process: Integration and Diagnosis 34
5.1 1. Reconstruct the dependency chain . . . 34
5.2 2. Reconstruct the minimum process . . . 35
5.3 3. Reconstruct the lifecycle . . . 35
5.4 4. Trace first run . . . 35
5.5 5. Trace yield and resume . . . 36
5.6 6. Trace exit . . . 37
5.7 7. Diagnose by concept . . . 37
5.7.1 Wrong stack . . . 37
5.7.2 Restart instead of resume . . . 38
5.7.3 Finished process runs again . . . 38
5.8 8. Function call versus process . . . 38
5.9 9. Why assembly is already disappearing . . . 39
5.10 10. CP1 behavioural contract . . . 39
5.11 11. Readiness questions . . . 40
5.12 12. Bridge to the next problem . . . 40

Course Overview

This site contains the teaching material for **BVS1 – Betriebssysteme** at TH Köln, ICCT, for **Wintersemester 2026/2027 · TH Köln · ICCT**. The course is attended by students from multiple Bachelor programmes.

The course provides two aligned representations: concise lecture-specific Beamer slides for classroom interaction, and a reader-oriented chapter source used for both the PDF script and the HTML course.

Course Material

The lecture chapters are listed in the HTML navigation and in the PDF table of contents.

1 From Executing Code to Operating Systems

2 From Executing Code to Operating Systems

An operating system is familiar enough that it is easy to begin with examples rather than with the problem it solves. Windows, Ubuntu, Android and macOS are recognizable operating-system environments, but naming familiar systems does not yet explain why operating systems exist.

BVS1 therefore starts with a smaller question:

Can a computer execute a program without an operating system?

The first practical system is deliberately tiny. An RV64 target machine runs one program in QEMU and prints:

```
Hello BVS1
```

The experiment gives us a concrete machine to reason about before we introduce operating-system mechanisms.

2.1 1. One program can run without a guest operating system

At the lowest level, a processor executes instructions. Those instructions operate on registers and memory and can communicate with devices.

One important idea of the stored-program / Von Neumann model is that **instructions and data are both kept in memory**.

A deliberately simplified machine model is:

```
     instructions + data


          memory


            CPU


          devices
```

This model omits many details of real hardware, but it is enough for the first distinction:

> **CPU, memory and devices exist independently of an operating system.**

The point here is conceptual rather than historical or microarchitectural. Modern processors may use separate instruction and data caches internally, but the important point here is that instructions and data both occupy target memory. Lecture 2 returns to this idea when the linker script makes the abstract memory model concrete.

In the `Hello BVS1` experiment, QEMU provides the target machine. The target contains an RV64 CPU, memory and devices. The program is loaded and begins executing without a guest operating system.

The physical computer still runs an operating system, and our Ubuntu development environment also runs an operating system. The important observation concerns the **target**:

> **Executing instructions is not, by itself, what makes an operating system necessary.**

2.1.1 Interpreting the experiment

The classroom experiment deliberately separates observation from explanation. Consider these statements:

- QEMU executed our program.
- A virtual CPU executed our program.
- Ubuntu executed our guest program.
- The target had an operating system.
- QEMU was already running before our guest program started.
- The string `Hello BVS1` existed somewhere in memory while the program was running.

The purpose of discussing them is to expose the execution model.

A precise description is:

- the Ubuntu development VM runs the QEMU process;
- QEMU implements the target machine;
- the target contains an emulated RV64 CPU, memory and devices;
- the RV64 program runs on that target;
- the target has no guest operating system.

Saying "QEMU executed the program" is understandable in informal language, but the more useful model is that QEMU **emulates the machine whose CPU executes the target instructions**.

2.2 2. The ISA connects software and processor

Machine code only has meaning if software and the processor agree on what each instruction means.

This agreement is the **Instruction Set Architecture (ISA)**.

An ISA specifies properties visible to machine-level software, including:

- instructions and their encodings,
- registers,
- memory-addressing rules,
- exceptional control transfers,
- privilege mechanisms.

Different processor implementations can implement the same ISA.

The ISA should therefore be distinguished from the **microarchitecture**:

- **ISA:** the contract visible to software;
- **microarchitecture:** how a particular processor implements that contract internally.

For BVS1, the teaching target uses **RV64**, the 64-bit form of RISC-V.

2.2.1 Why RV64?

RISC-V is a teaching choice, not a claim that it is the best ISA for every system.

It offers a comparatively regular architecture, public specifications, a modern privilege model, suitable development tools, and a well-supported QEMU `virt` machine. These properties let us expose operating-system mechanisms without making historical processor complexity the subject of the course.

2.3 3. Development host and target are different roles

The machine on which software is developed does not need to use the same ISA as the machine for which the program is built.

In BVS1:

- the **development host** is the Ubuntu `bvs1` VM in which the development tools run;
- the **target** is the RV64 machine for which we build the program.

The development VM may itself run on an x86-64 or Arm64 host platform. The teaching target is nevertheless RV64.

This gives two separate questions:

1. Where do the development tools execute?
2. Which machine will execute the generated program?

Keeping these roles separate is essential for understanding both cross-compilation and emulation.

2.4 4. Virtualization and emulation are different

The most useful first question is:

Can the physical CPU execute the guest or target instructions directly?

That question separates the roles of virtualization and emulation more clearly than the word *virtual machine* by itself.

2.4.1 Virtualization: the guest instructions already belong to the host ISA

In hardware-assisted virtualization, guest and host normally use the same processor ISA.

A guest instruction is therefore, in the ordinary case, already an instruction that the physical CPU understands. Most non-privileged guest instructions can execute directly on the physical processor.

The virtualization mechanism controls the parts that must not simply be handed to the guest, for example:

- isolation between machines,
- privileged processor state,
- access to memory and devices,
- transitions that require intervention by the virtualization layer.

A simplified model is:

```
guest instruction
       ↓
physical CPU
```

The important point is not that virtualization performs no mediation. It is that the guest machine does not require a different instruction vocabulary from the physical CPU.

In BVS1, Multipass manages the Ubuntu development VM using virtualization facilities available on the host platform. The exact backend can differ between operating systems.

2.4.2 Emulation: the target instruction may not be a host instruction

Emulation can reproduce a machine whose ISA differs from the host ISA.

Suppose the physical processor uses x86-64 while the BVS1 target uses RV64. The physical x86-64 CPU does not directly understand an RV64 instruction.

An emulator must therefore reproduce the effect of the target instruction. Conceptually:

```
RV64 target instruction
          ↓
        QEMU
          ↓
interpret / translate
          ↓
host operations
          ↓
physical CPU
```

The word **interpret** is useful for the conceptual distinction, but it should not be understood as a claim that an emulator must process every instruction individually. QEMU can dynamically translate blocks of target instructions into host code.

The essential point is:

> **Virtualization can largely run guest instructions directly; emulation must reproduce what target instructions would do when the host cannot execute them directly.**

2.4.3 Our BVS1 machine stack

The resulting stack is:

```
physical host
     ↓  virtualization (managed through Multipass)
Ubuntu development VM: bvs1
     ↓  runs
QEMU process
     ↓  emulates RV64
RV64 target machine
     ↓  executes
Hello BVS1
```

The layers have different roles:

- the **physical host** provides the real CPU, memory and devices;

- the **Ubuntu development VM** is the common environment in which the course tools run;
- **Multipass** manages that development VM and is not an additional machine layer;
- **QEMU** is a process running inside the development VM;
- QEMU emulates the **RV64 target machine**;
- `Hello BVS1` executes on that target.

Only the RV64 target is the teaching machine whose operating-system mechanisms we will build. It initially contains no guest operating system.

A **virtual machine** is the machine environment presented to software. Virtualization and emulation are different techniques that can provide such environments.

QEMU itself is capable of more than the configuration used here: in other settings it can also participate in hardware-assisted virtualization. In BVS1, its relevant role is RV64 emulation.

2.5 5. The toolchain connects development host and target

The source program is written in a human-readable programming language, but the target CPU needs RV64 machine instructions.

A simplified build path is:

```
source code
    ↓
cross compiler / assembler
    ↓
object files
    ↓
linker
    ↓
RV64 executable
    ↓
QEMU
    ↓
RV64 target machine
```

The cooperating compiler, assembler, linker and binary utilities form the **toolchain**.

A **cross compiler** runs on one architecture while producing code for another target architecture.

QEMU has a different role: it does not translate source code into the executable. It provides the machine on which the already generated target program can run.

2.5.1 Object files are not yet the complete program

Compilation and assembly can already produce target-specific object files. These can contain RV64 machine code, but they are not yet the complete executable. Separate pieces still have to be combined and references between them resolved.

The final result of this build path is a linked RV64 ELF executable that QEMU can load for the target machine.

Lecture 2 unfolds this path manually: we inspect the object files, discover what is still unresolved, and then see why linking and the target memory layout are necessary.

2.6 6. When does an operating system become useful?

The `Hello BVS1` target can run one tiny program without a guest operating system.

Now change the requirement:

> **The machine must run two programs.**

New questions immediately appear:

- Which program receives the processor?
- Where is each program's state kept?
- What happens when one program waits for a device?
- Can one program overwrite another program's memory?
- Who coordinates access to shared devices?
- How does a program request a shared service?

Increase the requirements further:

- many programs,
- multiple users,
- persistent storage,
- keyboard and display,
- networking,
- failures,
- programs that do not trust one another.

The problem is no longer merely "execute instructions".

Now the system needs mechanisms for **sharing, coordination, protection and abstraction**.

This gives us the working definition used in BVS1:

> An **operating system** is software that helps turn hardware into a usable computer system by providing abstractions, resource management, coordination, protection, and services for programs and users.

This is intentionally a working definition.

We will test and sharpen it as concrete mechanisms are introduced.

Two classical viewpoints are already visible:

1. **Operating system as resource manager**
   CPU time, memory, storage and devices are finite resources that must be shared and controlled.
2. **Operating system as abstract machine**
   Programs should not need to manipulate every hardware detail directly. The operating system can provide safer and more useful abstractions and services.

2.7 7. Applying the working definition

The opening lecture deliberately asks students to classify several familiar terms before the working definition is established.

After deriving the definition, the distinctions become clearer:

- **Windows, Ubuntu and Android** are complete operating-system environments.
- The **Linux kernel** is the core used by Linux-based operating-system environments, not such an environment by itself.
- **UEFI** is firmware and a firmware interface/environment.
- **QEMU** is an emulator and virtualizer.
- **Docker** provides container tooling built on operating-system mechanisms.
- **Firefox** is an application.

The purpose of the exercise is not to memorize labels.

It is to ask which responsibilities a component actually assumes and at which layer it operates.

Exercise 1 extends this idea with less obvious boundary cases such as a boot manager, Proxmox, and the deliberately provocative question “Is Emacs an operating system?”

2.8 8. From Hello BVS1 to a minimal operating system

The course deliberately starts with a target that can do very little:

```
Hello BVS1
```

The semester question is:

> **What must we add to Hello BVS1 before we would call the result an operating system?**

Initial answers may include:

- handling devices,
- multiple executions,
- scheduling,
- protection,
- memory management,
- system services,
- communication,
- persistent storage,
- users,
- a command interface.

At the beginning these are hypotheses, not a checklist to memorize.

We add mechanisms only when a new requirement gives us a reason for them.

The practical target remains:

> **a small educational operating system on RV64 and QEMU `virt`, eventually providing a simple shell**

The red thread is:

```
a machine executes code
    ↓
software communicates with devices
    ↓
events must be handled
    ↓
multiple programs must coexist
    ↓
programs compete for the CPU
    ↓
programs and resources need protection
    ↓
programs need operating-system services
    ↓
programs need coordination
    ↓
programs need communication
    ↓
a user needs to control programs
    ↓
a simple shell
```

The course is therefore not organized as a catalogue of terminology.

Each new mechanism should answer a problem that the current system has acquired.

2.9 9. Key ideas

Lecture 1 establishes a small number of distinctions that we will use repeatedly:

- A target processor can execute a program without a guest operating system.
- CPU, memory and devices exist independently of the OS.
- An ISA defines the machine-level contract on which software relies.
- Development host and target are different roles.
- Virtualization and emulation are different techniques.
- Multipass gives us the virtualized development environment; QEMU emulates our RV64 target in the BVS1 setup.
- A cross-toolchain produces machine code for a target ISA different from the development system.
- Operating systems become useful as requirements for sharing, abstraction, coordination, protection and services grow.
- The BVS1 system grows only when a new requirement motivates a new mechanism.

A useful test of understanding is to explain this complete path:

```
source code
    ↓
cross-toolchain
    ↓
RV64 executable
    ↓
QEMU process in the development VM
    ↓
emulated RV64 machine
    ↓
observable program behaviour
```

and then explain why successful execution still does **not** imply that the target contains an operating system.

2.10 References

- Remzi H. Arpaci-Dusseau and Andrea C. Arpaci-Dusseau, *Operating Systems: Three Easy Pieces*. https://pages.cs.wisc.edu/~remzi/OSTEP/
- Andrew S. Tanenbaum and Herbert Bos, *Modern Operating Systems*, Pearson.

- RISC-V International, RISC-V ISA specifications. https://docs.riscv.org/
- QEMU documentation, RISC-V `virt` Generic Virtual Platform. https://www.qemu.org/docs/master/system/riscv/virt.html
- Canonical Multipass documentation. https://documentation.ubuntu.com/multipass/

3 From Source to Process

Lecture 1 showed that an RV64 target can execute a small program without a guest operating system. Lecture 2 asks how that execution comes into existence and why, for an operating system, simply starting a program is not enough.

The path is:

```
source → objects → linked ELF → memory → _start → main
```

and then:

```
executing program → OS-managed execution → process
```

The build tools are therefore not the final topic. We unfold them once because they make the transition from source code to a running machine state visible.

3.1 1. From source code to target-specific object code

We first compile `main.c` without linking:

```
mkdir -p build/manual

riscv64-unknown-elf-gcc -march=rv64imac -mabi=lp64 \
  -mcmodel=medany -msmall-data-limit=0 -c main.c -o build/manual/main.o
```

The important option is `-c`: stop before linking.

Inspecting the result with

```
file build/manual/main.o
```

shows that `main.o` is already a RISC-V ELF object.

This distinction matters: the compiler has already translated C into code for our target ISA. The object file is therefore not architecture-neutral. But it is also not yet a complete

program. Separate pieces can still refer to one another, and their final addresses have not yet been established.

The startup assembly is translated in the same way:

```
riscv64-unknown-elf-gcc -march=rv64imac -mabi=lp64 \
  -mcmodel=medany -c start.S -o build/manual/start.o
```

3.2 2. Why object files still need a linker

Using `nm` ("name list") lets us inspect the symbols in the two objects:

```
riscv64-unknown-elf-nm build/manual/start.o
riscv64-unknown-elf-nm build/manual/main.o
```

For the symbols relevant here, we can read the result as:

```
start.o defines _start
start.o needs   main
start.o needs   _stack_top

main.o  defines main
```

The important concept is not the letters printed by `nm`, but the relationship between separately translated pieces. One object can provide a symbol while another refers to it. The linker connects those pieces.

The unresolved `_stack_top` also reveals a second responsibility of the linker: it must establish where the program lives in target memory.

3.3 3. From the stored-program idea to a concrete memory layout

Lecture 1 used the abstract model:

```
instructions + data
        ↓
      memory

       CPU
```

Our real program now contains different kinds of bytes that need concrete places:

```
RV64 memory from 0x80000000

+-----------------------------------------------------------+
| .text    _start, main, ...      | instructions          |
| .rodata  "Hello BVS1\n"         | constant data         |
| .data                           | initialized data      |
| .bss                            | zero-initialized data |
| C stack  (top: _stack_top)      | runtime data          |
+-----------------------------------------------------------+
```

These roles are not properties that memory itself knows. They arise from the executable layout and from how the program and CPU use the addresses.

The course-provided `linker.ld` makes that layout concrete:

```
ENTRY(_start)

MEMORY {
    RAM (rwx) : ORIGIN = 0x80000000, LENGTH = 128M
}

. = ALIGN(16);
. += 16K;
_stack_top = .;
```

Here `.` is the linker's current output address. After 16 KiB have been reserved for the initial stack, that address is named `_stack_top`.

Students do not need to write linker scripts in BVS1. The important idea is that the linker does not merely join files; it constructs one final address space in which the pieces can refer to one another.

3.4 4. Linking produces the program QEMU can load

We link the two objects using the course layout:

```
riscv64-unknown-elf-gcc -nostdlib -nostartfiles -no-pie \
  -T linker.ld build/manual/start.o build/manual/main.o \
  -o build/manual/hello-bvs1.elf
```

At this point the linker has:

- combined the objects;

- resolved the symbol references;
- established the final memory layout.

The resulting ELF is what QEMU receives. QEMU does not compile or link the program; it provides the emulated RV64 machine and loads the already-built image.

3.5 5. Why execution starts before `main`

The linker script declares:

```
ENTRY(_start)
```

and `_start` is defined in `start.S`:

```
_start:
    la   sp, _stack_top
    call main
1:
    j    1b
```

This small fragment exposes an important boundary between machine startup and ordinary C code.

Before `main` can run, startup code establishes at least the execution environment that C expects. Here that means setting `sp`, the stack pointer, to the address denoted by `_stack_top`.

Then `call main` transfers control to `main` and remembers where execution should return. We deliberately do not open the full calling convention yet. The fact that a return point must somehow be preserved becomes important in Lecture 3.

The local label `1:` and `1b` only express the final fallback loop: if `main` returns, execution remains there.

3.6 6. The whole path, and what Make hides

Running the ELF manually:

```
qemu-system-riscv64 -machine virt -m 128M -smp 1 -nographic \
  -bios none -kernel build/manual/hello-bvs1.elf
```

reconstructs the complete path:



Compilation, assembly, linking, and QEMU itself run in the Ubuntu development VM. The final RV64 instructions execute in the emulated target.

Make automates this workflow. It decides when a target is out of date and runs the corresponding recipe. The GCC driver performs the translation steps and invokes the linker. QEMU provides the machine on which the result executes.

Doing the path manually once is useful because the automation then has a visible meaning rather than being a black box.

3.7 7. Change, rebuild, observe

If only the string in `main.c` changes, we can predict the consequences before rebuilding:

- `main.o` must change;
- `start.o` does not;
- the final ELF must be linked again;
- `linker.ld` does not change;
- QEMU itself does not change.

This small exercise is about maintaining a causal model of the system:

predict → change → build → run → observe → explain

That pattern will be reused when the changes are no longer only strings, but operating-system mechanisms.

3.8 8. Why an executing program is not yet a process

Our current target has one execution:

```
_start
  ↓
main
  ↓
print
  ↓
for (;;) {}
```

Once it starts, nothing inside our target can currently stop it, regain control, and later continue it.

That is the new requirement. An operating system must be able to manage an execution rather than merely start it.

For our first model:

> A **process** is an execution that the operating system can manage.

For Checkpoint 1, "manage" means only:

```
create → run → yield → resume → exit
```

This is intentionally much smaller than a complete Unix process model. We do not yet need address spaces, files, process IDs, or scheduling policies.

The minimum lifecycle is:

```
create  → READY  run / resume → RUNNING  exit → FINISHED
            ↑
                  yield
```

A yielded process is still runnable. A finished process is not.

3.9 9. Yield, exit, resume, and restart

Yield and exit both transfer control back to the operating system, but they have different meanings.

- **yield**: the execution is not finished and must be able to continue later;
- **exit**: the execution is finished and must never continue.

The distinction becomes visible in this intended interface:

```
uart_puts("A\n");
process_yield();
uart_puts("B\n");
process_exit();
```

`process_yield()` and `process_exit()` are not library functions we already have. They are functions our minimal operating system still has to implement.

After `process_yield()`, the operating system has control. When the process runs again, it must continue with `uart_puts("B\n")`. Printing `A` again would mean that we restarted the execution instead of resuming it.

This is the conceptual step that makes the next lecture necessary.

3.10 10. The question that leads to execution context

To resume correctly, something about the paused execution must survive:

```
process runs
     ↓
yield
     ↓
OS regains control
     ↓
resume
     ↓
continue exactly where it stopped
```

What must be preserved so this can work?

At this point it is enough to suspect that some machine state must survive the pause. Lecture 3 determines exactly what that state is.

The final path of Lecture 2 is therefore:

```
source → objects → linked ELF → memory → _start → main
```

followed by:

```
executing program → OS-managed execution → process
```

The remaining question is:

> **What machine state makes resume possible?**

4 Preserving Execution

Checkpoint 1 is intentionally modest:

> **I have a process.**

Before a process can exist as an operating-system abstraction, the system needs one simpler capability: execution must be able to stop and later continue.

This chapter develops only the machine state needed to make that idea visible.

4.1 1. From “run” to “continue later”

The current `Hello BVS1` program starts and then follows one continuous control flow.

Now imagine:

```
do_first_thing();

process_yield();

do_second_thing();
```

The intended behaviour is:

1. execute `do_first_thing()`;
2. enter `process_yield()`;
3. let the kernel regain control;
4. later resume the process;
5. continue with `do_second_thing()`.

The process must therefore retain enough machine state to preserve its continuation.

Conceptually, execution state includes:

- where execution should continue;
- register values that are required to survive;
- the current stack position;
- the stack contents themselves.

For the first checkpoint, we deliberately exploit normal C function-call rules so that the saved context stays small and understandable.

4.2 2. Return addresses make continuation visible

Consider:

```
void first(void)
{
    second();
    third();
}
```

A machine-level function call must know both where to go and where to return afterwards.

RISC-V uses the register `ra` as the return-address register.

At a conceptual level:

```
call second
    ↓
remember continuation in ra
    ↓
execute second
    ↓
ret
    ↓
continue using ra
```

The exact instruction encoding is not important here.

The relevant insight is that a continuation is represented by machine state.

4.3 3. Why a stack is needed

Nested calls require values to survive while other functions execute.

The stack provides storage whose lifetime follows nested function calls.

RISC-V software conventionally uses `sp` as the stack pointer, and the stack grows toward lower addresses.

If two independent executions have different stacks, they also need different saved values of `sp`.

For example:

```
execution A: sp = 0x81003f80
execution B: sp = 0x81007f40
```

Restoring B's registers but accidentally retaining A's stack pointer would cause B to use the wrong stack.

The stack pointer is therefore part of the context that must be preserved.

4.4 4. The calling convention reduces what we need to save

A normal C function call does not promise that every general-purpose register is unchanged afterwards.

The calling convention distinguishes, among other categories:

- **caller-saved registers** — the caller must preserve them if it needs them later;
- **callee-saved registers** — the called function must restore them before returning.

RISC-V names the callee-saved general-purpose registers `s0` through `s11`.

For a cooperative context switch that occurs at a normal function-call boundary, we can exploit that contract.

A minimal saved context can therefore contain:

```
ra
sp
s0 ... s11
```

The reason for saving `ra` here deserves care. In the general ABI it is not a callee-saved register. In the context-switch routine it is saved because it identifies the continuation to which the restored context should return.

This is not yet a full interrupt or trap frame. Later preemption occurs at an asynchronous boundary and requires different handling.

4.5 5. Represent the saved context in C

A compact representation is:

```
struct context {
    uint64_t ra;
    uint64_t sp;
    uint64_t s[12];
};
```

The exact final layout must agree with the assembly offsets used by the context-switch scaffold.

The important conceptual distinction is:

> A context is saved execution state. It is not yet the whole process.

The structure simply answers:

> **Where, on which stack, and with which preserved register values can this execution continue?**

4.6 6. The narrow assembly boundary

A simplified context switch can be read as:

```
switch_context:
    sd ra, 0(a0)
    sd sp, 8(a0)

    ld ra, 0(a1)
    ld sp, 8(a1)
    ret
```

The calling convention passes:

```
a0 → address of context to save
a1 → address of context to restore
```

The complete scaffold also preserves the callee-saved `s` registers.

The learning target is not to reproduce the whole routine from memory.

The important sequence is:

1. store the current continuation and stack position into one context;
2. load another context's continuation and stack position;
3. execute `ret`;
4. continue according to the restored `ra` on the restored stack.

This is the key conceptual moment:

> The final `ret` returns into the restored context, not necessarily to the code that called the switch most recently in wall-clock time.

4.7 7. Kernel context and process context

Checkpoint 1 needs two saved continuations:

```
kernel context

switch_context

process context
```

When the kernel first switches to the process, the kernel context is saved.

When the process later yields, the process context is saved and the kernel context is restored.

When the kernel resumes the process, the reverse happens again.

The kernel context is not being introduced here as a second user process. It is simply the state needed so kernel code can continue after control returns from the process.

For CP1, this is not yet a user/kernel privilege boundary.

4.8 8. Starting a context that has never run

A previously running context already has a saved continuation.

A new process does not.

We solve that by constructing an initial context whose values make the normal restore mechanism enter a bootstrap function:

```
sp → top of the process's new stack
ra → process_bootstrap
```

Suppose the process has stack storage:

```
uint8_t stack[4096];
```

Because the stack grows downward, its initial stack pointer begins near the top of that storage.

The exact alignment details can be handled by provided code or a helper. The important idea is that the process owns separate stack storage and that the saved context points into it.

4.9 9. The bootstrap returns us to ordinary C

A small bootstrap can conceptually look like:

```
static void process_bootstrap(void)
{
    current->entry();
    process_exit(0);
}
```

The first restore enters this function because the initial context contains its address in `ra`.

From there, ordinary C function calls can be used.

This is an important pattern for the course:

> Assembly is used only at the narrow boundary where the abstraction needs direct machine control. Once the boundary has been crossed, ordinary C and higher-level operating-system concepts take over.

4.10 10. Trace a complete cooperative switch

Assume the kernel is running and a new process has:

```
process.context.sp → its stack
process.context.ra → process_bootstrap
```

The kernel calls:

```
switch_context(&kernel_context, &process.context);
```

The switch:

1. saves the kernel continuation and stack;
2. restores the process context;
3. executes `ret`;
4. enters `process_bootstrap`.

Later the process calls:

```
process_yield();
```

Inside that operation, the process context is saved and the kernel context is restored.

The switch then returns to the kernel continuation saved by the earlier switch.

When the kernel later restores the process context, execution resumes inside the previous call to `process_yield()`. That function then returns normally, and the process continues after its yield.

This is what "resume" means in the first process implementation.

4.11 11. From saved context to first process

A saved context answers:

> Where and with which machine state can execution continue?

Checkpoint 1 needs one slightly larger abstraction:

```
struct process {
    struct context context;
    uint8_t stack[4096];
    void (*entry)(void);
    enum state state;
    int exit_status;
};
```

This is one useful teaching representation, not a required internal design.

The process adds:

- an entry function;
- owned stack storage;
- lifecycle state;
- an exit status.

No scheduler, address space, or user-mode boundary is required.

4.12 12. Minimal lifecycle

A small lifecycle is enough:

```
unused
   ↓ create
ready
   ↓ run
running
   ↓ yield
ready
   ↓ run
running
   ↓ exit
finished
```

The distinctions are semantic:

- **create** prepares an executable context but does not run it;
- **yield** pauses a process that may run again;
- **exit** finishes a process that must not run again.

4.13 13. Creation prepares the initial context

Creation establishes:

```
entry function
process stack
context.sp → top of that stack
context.ra → process_bootstrap
state → ready
```

The entry function itself has not executed yet.

The first run uses the same context-switch operation that later performs resume.

This is why the artificial initial `ra` and `sp` matter: they make a never-before-run process satisfy the same restore interface as an already-paused process.

4.14 14. Yield and resume

Suppose the process body is:

```
first_part();
process_yield();
second_part();
```

Yield means:

```
running → ready
save process context
restore kernel context
```

The kernel may perform other work and later run the same process again:

```
ready → running
save kernel context
restore process context
```

The restored continuation returns into the old `process_yield()`, which then returns normally. Execution proceeds with `second_part()`.

No scheduler is required yet. Kernel code can explicitly decide to run the one process again.

4.15 15. Exit

Exit has different semantics:

```
process_exit(42);
```

means:

```
running → finished
exit_status = 42
restore kernel context
do not resume this process again
```

The saved context may still physically exist. The process abstraction determines whether the operating system is allowed to use it.

This distinction between machine capability and OS policy becomes more important later.

4.16 16. The full CP1 behaviour

The first checkpoint can now be described completely:

```
create
→ run on own context / stack
→ yield
→ kernel regains control
→ resume
→ exit with status
→ kernel regains control
```

All concepts required by that sequence have been introduced by the end of Lecture 3.

Lecture 4 therefore serves as consolidation, diagnosis, and transfer rather than adding another required mechanism immediately before the earliest checkpoint.

4.17 17. Key ideas

Students should leave this chapter able to explain:

- why execution cannot resume unless relevant machine state survives;
- why the stack pointer and separate stack storage matter;
- how normal calling conventions reduce the state needed at a cooperative switch boundary;
- why `ra` can serve as the cooperative continuation;
- how a small assembly switch saves one context and restores another;
- how an artificial initial `ra` and `sp` start a context that has never run before;
- how a context becomes a process by adding entry, stack ownership, lifecycle state and completion information;
- why create, yield, resume and exit are different operations;
- why the assembly boundary can remain small while the process abstraction itself is built mostly in C.

5 I Have a Process: Integration and Diagnosis

All concepts required for the first checkpoint have already been introduced before this chapter.

The purpose here is different:

> **Can the complete mechanism be reconstructed, explained, and diagnosed without following a recipe?**

This is the final consolidation step before:

> **CP1 — I have a process.**

5.1 1. Reconstruct the dependency chain

The first process depends on several layers that were introduced separately:

```
source code
   ↓
RV64 executable
   ↓
machine execution
   ↓
saved execution context
   ↓
separate stack
   ↓
process lifecycle
   ↓
observable yield / resume / exit
```

The lower layers are still real, but the process abstraction lets ordinary reasoning move upward.

A useful distinction is:

- **machine-level mechanism** — saved registers, stack pointer, return address, context switch;
- **operating-system abstraction** — process state, lifecycle, entry, completion status.

5.2 2. Reconstruct the minimum process

Without relying on one required source-code layout, a first process must answer:

- where can execution continue?
- which stack belongs to it?
- what code should it run?
- may it run again?
- how did it finish?

A saved context answers only the first question and part of the second.

The broader process abstraction owns the stack storage, identifies the entry function, records lifecycle state, and retains an exit status.

5.3 3. Reconstruct the lifecycle

The intended CP1 lifecycle is:

```
unused
   ↓ create
ready
   ↓ run
running
   ↓ yield
ready
   ↓ run / resume
running
   ↓ exit
finished
```

Creation prepares but does not execute.

Yield means the process may continue later.

Exit means the process is complete and must not resume.

This distinction must remain true even if a stale saved context still physically exists in memory.

5.4 4. Trace first run

A fresh process has an artificial initial context:

```
sp → top of its own stack
ra → process_bootstrap
state = READY
```

When the kernel switches to it:

1. the kernel context is saved;
2. the process context is restored;
3. the final return uses the restored `ra`;
4. the bootstrap begins on the process stack;
5. ordinary C process code starts.

The initial continuation was constructed during process creation rather than saved from previous process execution.

5.5 5. Trace yield and resume

Suppose:

```
puts("A\n");
process_yield();
puts("B\n");
```

A correct observable trace can include:

```
A
<kernel output>
B
```

This demonstrates that the process continued after the yield.

A trace such as:

```
A
<kernel output>
A
```

suggests restart or reinitialization rather than resume.

At yield:

```
RUNNING → READY
save process context
restore kernel context
```

At resume:

```
READY → RUNNING
save kernel context
restore process context
```

The restored return address eventually lets `process_yield()` return normally.

5.6 6. Trace exit

If the process calls:

```
process_exit(42);
```

the required semantics are:

```
RUNNING → FINISHED
exit_status = 42
kernel regains control
process is not run again
```

The exit status is an observable completion result. It will later become useful when higher-level mechanisms such as a shell start processes.

5.7 7. Diagnose by concept

5.7.1 Wrong stack

Symptom:

```
process runs
process yields
kernel performs work
resume crashes or corrupts state
```

If the restored continuation looks plausible, investigate the process stack and saved `sp`.

A correct continuation on the wrong stack is still an invalid execution state.

5.7.2 Restart instead of resume

Expected:

```
process: first
kernel: back
process: second
```

Actual:

```
process: first
kernel: back
process: first
```

Likely conceptual causes include:

- the yielded continuation was not restored;
- the process was recreated/reinitialized before the second run.

5.7.3 Finished process runs again

If the kernel observes `FINISHED` and then resumes the same process, the lifecycle abstraction has been violated.

The lower-level context-switch mechanism may still be physically capable of restoring the saved registers. The OS abstraction must prevent that operation.

5.8 8. Function call versus process

Compare:

```
hello();
```

with:

```
create hello
run
yield
kernel work
resume
exit
kernel observes status
```

The second case has independently resumable execution state.

Useful distinguishing properties are:

- separate stack storage;
- saved continuation;
- suspension without unwinding the call chain;
- explicit lifecycle state;
- observable completion status;
- operating-system control over when execution continues.

A particular function name or file structure is not what makes it a process.

5.9 9. Why assembly is already disappearing

At the machine boundary, the implementation still needs operations such as:

```
save registers
restore registers
ret
```

But above that boundary the useful vocabulary is now:

```
create process
run process
yield
resume
exit
```

This is intentional.

The course is climbing an abstraction hierarchy. Once the mechanism below an abstraction has been understood, students should be able to reason with the abstraction without repeatedly expanding it into assembly.

They must still be able to descend one layer when explanation or diagnosis requires it.

5.10 10. CP1 behavioural contract

A public checker should eventually establish behaviour equivalent to:

```
process starts
process yields
kernel regains control
process resumes
process exits with a known status
kernel observes completion
```

The checker should observe behaviour rather than enforce one private source-code architecture.

Passing an output checker alone is not evidence of understanding. At the mandatory checkpoint, each student must also explain the mechanism and handle a small equivalent variation, prediction, or diagnosis.

5.11 11. Readiness questions

Before the checkpoint, a student should be able to answer without following a recipe:

1. What state survives a yield?
2. Why does the process need its own stack?
3. How does first start differ from resume?
4. Why is exit not another yield?
5. How can output demonstrate resume rather than restart?
6. Which narrow assembly mechanism supports the higher-level C abstraction?
7. What would you inspect first if resume fails after kernel code has run?

These questions stay inside the announced competence family.

5.12 12. Bridge to the next problem

With one process, the kernel may explicitly decide when to run it again.

Now suppose:

```
process A never yields
process B is ready
```

If A keeps the CPU forever, B cannot progress.

That creates the next operating-system requirement:

> **How can the OS regain control when the running process does not cooperate?**

Timer events, preemption and scheduling belong to the next phase, not to CP1.